FPS MOBILE APPLICATION SDK

Integration Specification

Revision History

Date	Version	Changelog
10/2024	1.0.0	SDK Initial
01/2025	1.0.4	Integrating Flutter + Enhancing data collected
02/2025	1.0.4-a	● Fix bugs ● Enhance Flutter SDK: ○ Integrating iOS ○ In section "**Add the SDK to your project**" -> Remove "Include AAR" step ○ In section "**Override Your Activity Lifecycle Methods**" -> Modify "~~AppLifecycleState.paused~~" condition on app pause.
04/2025	1.0.5	Fix minor bugs: ● SDK returns non-arabic numbers on special locale (ex: Burmese)

Table of Contents

Revision History **1**
Table of Contents **2**
1. Introduction **3**
a. Purpose 3
b. Scope 3
c. Supported Platforms 3
2. System Architecture **5**
3. SDK Sequence Diagram **6**
4. Integration Guide **7**
a. Android (Kotlin + Java) 7
SDK Methods 7
onCheckFPSTrans 7
onResetFPSLastLiveTime 8
Step 1. Add the SDK to Your Project 8
Step 2. Initialize the SDK with the Service Layer 8
Step 3. Override Your Activity Lifecycle Methods 10
Step 4. Set the login action 11
b. Flutter 12
Step 1: Add the SDK to your project 12
Step 2. Initialize the SDK with the Service Layer 13
Step 3. Override Your Activity Lifecycle Methods 14
Step 4. Set the login action 15
5. FPS Gateway API **16**
a. Base URL 16
b. Endpoint 16
c. Header 16
d. Body 16
e. Response 16

1. Introduction

FPS Mobile Integration system is designed to identify and mitigate fraudulent activities carried out by malicious users on mobile devices. The system leverages advanced algorithms, data analysis, and behavioral tracking to detect unusual patterns that deviate from legitimate user behavior.

a. Purpose

FPS Mobile Application SDK empowers mobile applications with advanced fraud prevention capabilities by collecting unique device identifiers and information. Using sophisticated device fingerprinting, FPS Mobile helps detect and flag suspicious activities by capturing specific attributes that distinguish individual devices. This allows for enhanced security measures that prevent unauthorized access and fraudulent actions within your app. This is essential information for risk control looking to maintain platform integrity, secure user data, and ensure a safer, more trustworthy user experience.

b. Scope

FPS Mobile Application SDK is designed to provide a comprehensive solution for mobile fraud prevention by leveraging unique device identifiers. Its primary scope includes:

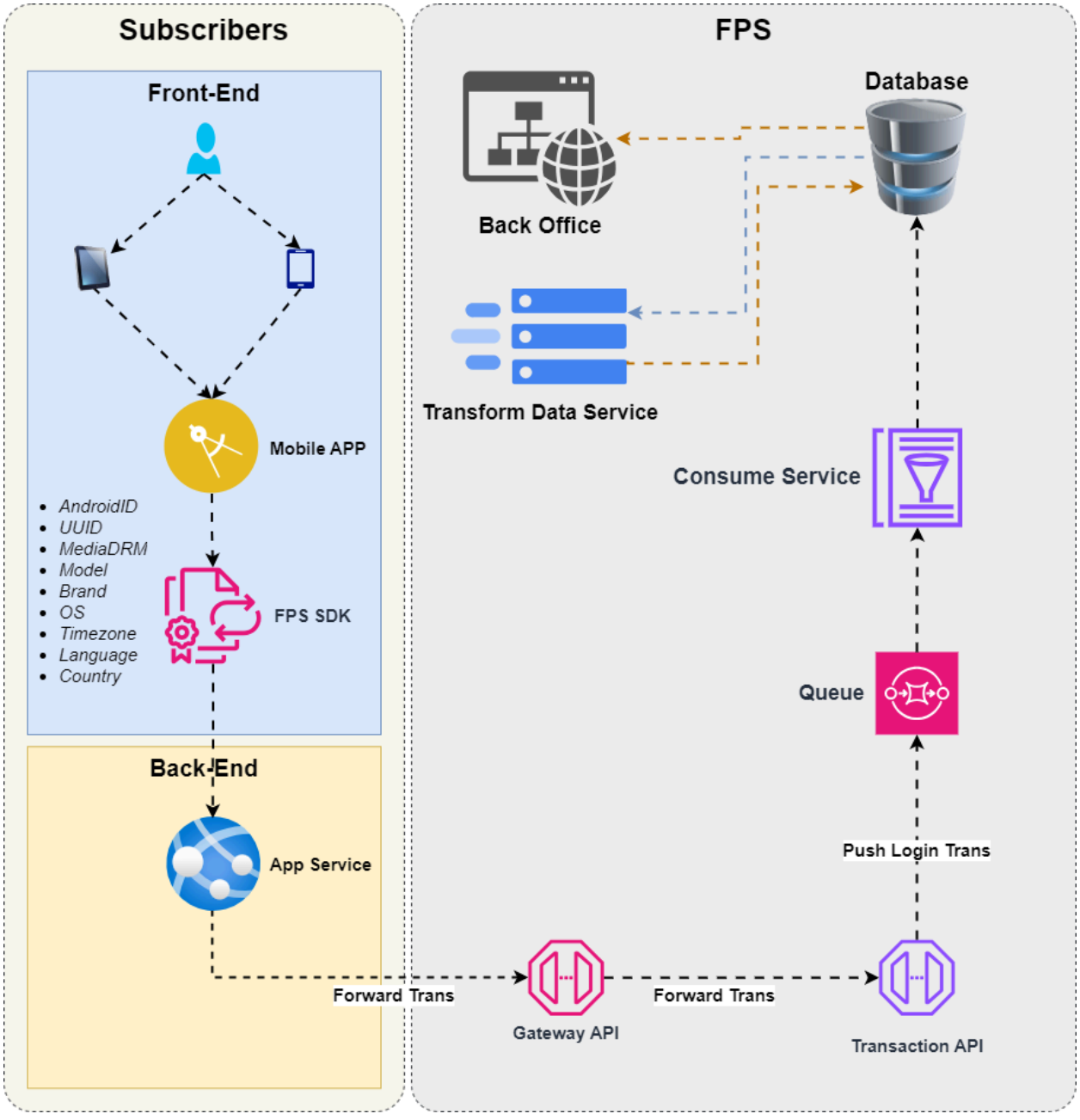
- **Device Identification:** Collects unique attributes from mobile devices to establish a distinct device identity, aiding in the recognition of returning users and detecting potential impersonation.
- **Fraud Detection:** Uses a blend of device-specific information to help detect suspicious or unusual activity patterns, enabling early identification of fraudulent behaviors within the app.
- **Security Enhancement:** Integrates seamlessly into mobile applications to reinforce security protocols, creating an additional layer of defense against unauthorized access and high-risk transactions.
- **User Experience Integrity:** Helps maintain a secure and trustworthy environment for users by reducing fraud-related disruptions, ensuring an authentic and seamless experience.

With **FPS Mobile Application SDK**, you can protect their applications against a range of fraud attempts by making use of detailed device information, thus safeguarding both app security and user trust.

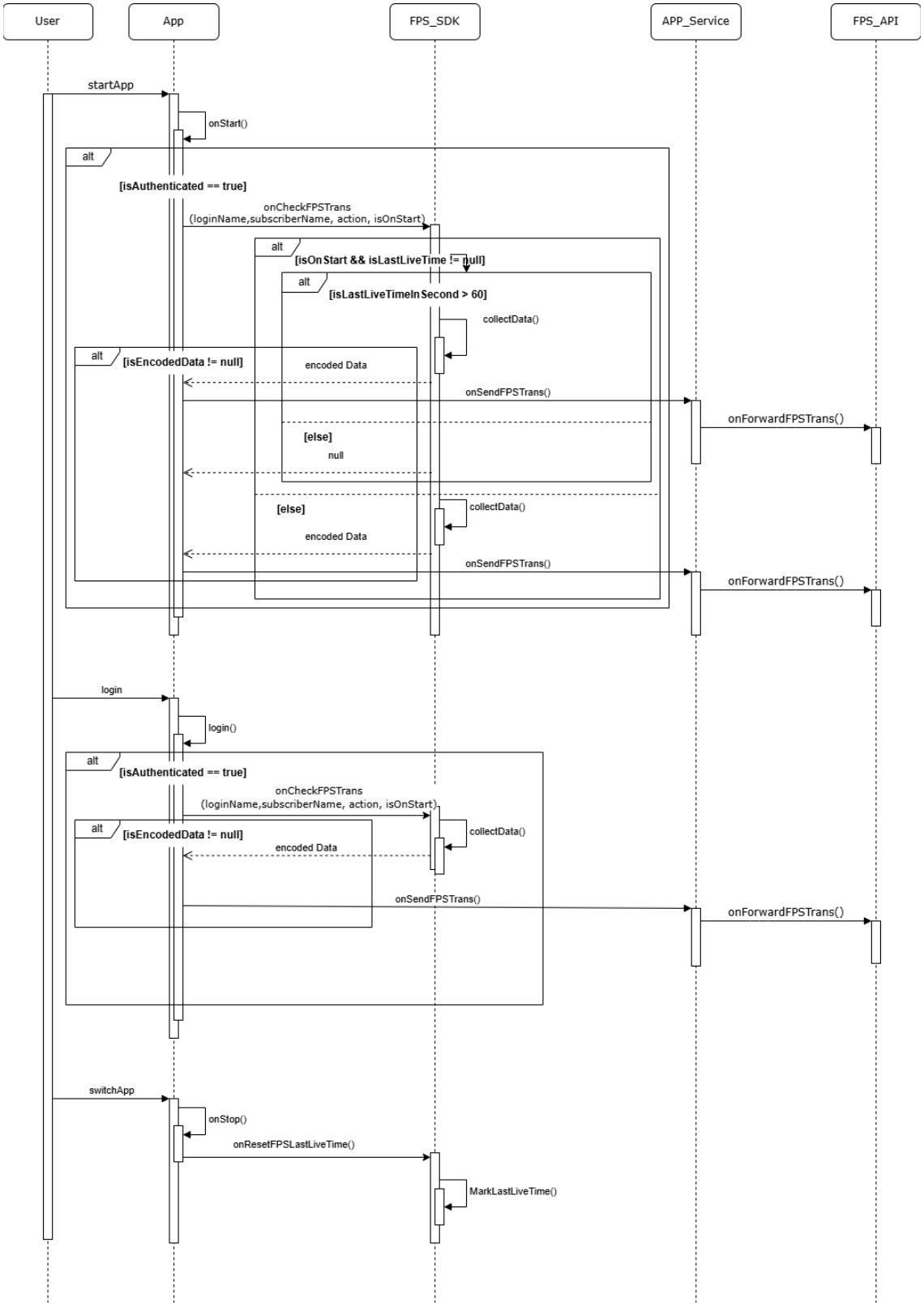
c. Supported Platforms

Platform	Language	Version	Description
Android	Kotlin + Java	Android 7 (API 24) - Android 13 (API 34)	
Flutter	Dart	Flutter 3.10 or higher Android 7 (API level 24+) or higher iOS	

2. System Architecture



3. SDK Sequence Diagram



4. Integration Guide

a. Android (Kotlin + Java)

This guide will walk you through the steps you need to take to start securing with FPS. Start by learning how to integrate the **FPS SDK** into your app and then move on to how to set up your app.

SDK Methods

onCheckFPSTrans

This method is going to collect device information & return an encoded message, which would **be forwarded to the FPS via App Service**. It should be run in asynchronous IO, preventing it from blocking the User Experience.

Method Signature

```
onCheckFPSTrans(context, subscriber, username, action, isOnStart)
```

Returns

String?

Parameters

Name	Type	Description	
context	Context	Application Context	
subscriber	String	The licensee's name was provided by the FPS team	Maxbet
username	String	The username that the client uses to log in	test001
action	String	The action where fps integrated	- **"login"** on login action - **"resume app"** when switch app - deposit, withdraw, etc…
isOnStart	Boolean	Check if the action was on Start activity	- **true** if action on onStart - **false** if otherwise

onResetFPSLastLiveTime

This method would help FPS get more precious transactions. It should be called in the **onStop (Lifecycle Activity) method** when the user navigates to another application.

Method Signature

```
onResetFPSLastLiveTime()
```

Returns

Unit

Step 1. Add the SDK to Your Project

1. Get **FPS SDK Package**, which includes 2 *.AAR files:
   - **com.fps.android.core-x.x.x.aar**
   - **com.fps.android.sdk-x.x.x.aar**

   -x.x.x means the current version of SDK.
2. Import the *.AAR file as a library project.
3. If using Android Studio, copy the *.AAR files and add them as a dependency to your module.
4. Go to **File ➢ New ➢ New Module ➢ Import *.AAR** and navigate to the location where the *.AAR file has been copied.
5. Make sure you add the following to your **build.gradle** file under the **dependencies** section:

```
dependencies {

    ...
    implementation(files("libs/com.fps.android.core-x.x.x.aar"))
    implementation(files("libs/com.fps.android.sdk-x.x.x.aar"))
}
```

Optional: Manual JAR Integration

If you are using **com.fps.android.core-x.x.x.jar** and **com.fps.android.sdk-x.x.x.jar** in your project, you can still use the same configuration.

1. Rename the new com.fps.android.core-x.x.x.aar to com.fps.android.core-x.x.x.zip.
2. Rename the new com.fps.android.sdk-x.x.x.aar to com.fps.android.sdk-x.x.x.zip.
3. Extract the classes.jar
4. Rename it to com.fps.android.core-x.x.x.jar and com.fps.android.sdk-x.x.x.jar .

Step 2. Initialize the SDK with the Service Layer

To initialize the SDK, follow these steps:

1. Create a Service class using the library to make an HTTP request

2. Define a function that calls **onCheckFPSTrans** method of SDK. This method is going to collect device information & return an encoded message, which will **be forwarded to the FPS via your App Service**. It should be run in asynchronous IO, preventing it from blocking the User Experience.

```
    interface IApiService {
        @Headers("Content-Type: application/json")
        @POST
        suspend fun send(@Url endpoint: String, @Body body: String):
String
    }

    class FpsService(
        baseUrl: String,
        private val endpoint: String
    ) {
        private val retrofit: Retrofit = Retrofit.Builder()
            .addConverterFactory(ScalarsConverterFactory.create())
            .baseUrl(baseUrl)
            .build()

        private val apiService: IApiService =
retrofit.create(IApiService::class.java)

        fun checkTransaction(
            context: Context,
            subscriber: String,
            username: String,
            action: String,
            isOnStart: Boolean = false
        ) {
            CoroutineScope(Dispatchers.IO).launch {
                try {
                    val fpsTrans = FPS.onCheckFPSTrans(context,
subscriber, username, action, isOnStart)

                    send(fpsTrans)
                }
                catch (e: HttpException) {
                    Log.d("HttpException", e.message!!)
                }
                catch (e: Exception) {
                    Log.d("ApiService", e.message!!)
                }
```

```
            }
        }

        private suspend fun send(transaction: String?) {
            if(transaction == null)
               return

            val jsonObject = JSONObject()
            jsonObject.put("transaction", transaction)

            apiService.send(endpoint, jsonObject.toString())
        }
    }
```

Step 3. Override Your Activity Lifecycle Methods

Application Lifecycle

Override the **onStop()**, **onStart()** methods in each of your activities to call the corresponding SDK methods as follows:

```
override fun onStart() {
            super.onStart()

            ...

            val model: LoginViewModel by viewModels()
            if(model.isAuthenticated()) {
                val username = model.getUsername()

                val fpsService = FpsService("APP_SERVICE_BASE_URL",
"APP_SERVICE_END_POINT")

                fpsService.checkTransaction(this, subscriber, username, "resume
app", true)
            }

        }

        override fun onStop() {
            super.onStop()

            ...

            FPS.onResetFPSLastLiveTime()
        }
```

Step 4. Set the login action

- FpsService should be called in **login methods**

```
fun login(
   context: Context
) {

   ...

   val fpsService = FpsService("APP_SERVICE_BASE_URL",
"APP_SERVICE_END_POINT")

   fpsService.checkTransaction(this, subscriber, username, "login", false)
}
```

b. Flutter

Step 1: Add the SDK to your project

- **Integrate as Flutter Plugin**
    1. Get **Flutter Plugin SDK**, which includes 2 folders named “lib” and “android”.
    2. Make sure you add the following to your **pubspec.yaml** file under the **dependencies** section:

```
dependencies:
 flutter:
   sdk: flutter

 fps:
   path: ../path/to/fps/flutter/sdk
```

Step 2. Initialize the SDK with the Service Layer

To initialize the SDK, follow these steps:

1. Create a Service class using the library to make an HTTP request
2. Define a function that calls **onCheckFPSTrans** method of SDK. This method is going to collect device information & return an encoded message, which will **be forwarded to the FPS via your App Service**. It should be run in asynchronous IO, preventing it from blocking the User Experience.

```
import 'dart:convert';

import 'package:flutter/cupertino.dart';
import 'package:fps/fps.dart';
import 'package:http/http.dart' as http;

class FpsService {
  String baseUrl;
  String endpoint;

  FpsService(this.baseUrl, this.endpoint);

  Future<String> checkTransaction(
      String subscriberName,
      String userName,
      String action,
      {bool isOnStart = false}
      ) async {
```

```
    final url = Uri.parse(baseUrl + endpoint);

    try {
      Fps fpsSDK = Fps();
      String? fpsTrans = await fpsSDK.onCheckFPSTrans(subscriberName, userName,
action, isOnStart: isOnStart);
      debugPrint(fpsTrans);
      final Map<String, dynamic> requestData = {
        'transaction': fpsTrans
      };
      if(isOnStart) {
        Future.delayed(const Duration(seconds: 3));
      }
      final response = await http.post(
        url,
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode(requestData),
      );
      return "checkTransaction: $baseUrl ${response.statusCode}";
    }
    catch (e) {
      debugPrint('FPS Error: $e');
      return "";
    }
  }
}
```

Step 3. Override Your Activity Lifecycle Methods

```
class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  _MyAppState createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> with WidgetsBindingObserver {
  AppLifecycleState? _lastState;
  @override
```

```
void initState() {
  super.initState();
  WidgetsBinding.instance.addObserver(this);
}

@override
void dispose() {
  WidgetsBinding.instance.removeObserver(this);
  super.dispose();
}



@override
void didChangeAppLifecycleState(AppLifecycleState state) {
  final viewModel = Provider.of<LoginViewModel>(context, listen: false);

  super.didChangeAppLifecycleState(state);

  if (Platform.isAndroid) {
    _handleAndroidLifecycle(state, viewModel);
  } else if (Platform.isIOS) {
    _handleIOSLifecycle(state, viewModel);
  }
}

void _handleAndroidLifecycle(
    AppLifecycleState state, LoginViewModel viewModel) {
  if (state == AppLifecycleState.resumed) {
    _handleResumedState(viewModel, "resume app");
  } else if (state == AppLifecycleState.paused) {
    _resetFpsLastLiveTime();
  }
}

void _handleIOSLifecycle(AppLifecycleState state, LoginViewModel viewModel) {
  if (state == AppLifecycleState.resumed) {
    _handleResumedState(viewModel, "resume app");
  } else if (state == AppLifecycleState.inactive &&
      _lastState == AppLifecycleState.resumed) {
    _resetFpsLastLiveTime();
```

```
    }

    _lastState = state;
  }

  void _handleResumedState(LoginViewModel viewModel, String action) {
    if (viewModel.isAuthenticated) {
      String userName = viewModel.userName;
      String subscriber = viewModel.subscriber;

      FpsService fpsService =
          FpsService("APP_SERVICE_BASE_URL", "APP_SERVICE_END_POINT");
      fpsService.checkTransaction(subscriber, userName, action,
          isOnStart: true);
    }
  }

  void _resetFpsLastLiveTime() {
    Fps fpsSDK = Fps();
    fpsSDK.onResetFPSLastLiveTime();
  }
```

Step 4. Set the login action

- FpsService should be called in **login methods**

```
Future<String> login(BuildContext context) async {
    FpsService fpsService = FpsService("APP_SERVICE_BASE_URL",
"APP_SERVICE_END_POINT");
    return fpsService.checkTransaction(subscriber, userName, "login");
  }
```

5. FPS Gateway API

This is an API to post the transaction to FPS. After the SDK returns an encoded message, this message will be sent to your Backend Service. There, you can capture an IP Address in an HTTP request from a Mobile App and attach it in the header **"Fps-Forwarded-For"** with the message in the body, then forward it to FPS Gateway.

a. Base URL

i. UAT

- External network: `https://fpt-uat.starixuat.com:6012`
- Internal network: `http://fpt-uat.nebula.prod`

ii. Production

- External network: `https://api.detecas.com`
- Internal network: `http://fpsapi.nebula.prod`

b. Endpoint

```
POST /FPS/MobileCheckTransaction
```

c. Header

Name	Description
Fps-Forwarded-For	The user's IP that is captured from the App Service

d. Body

Content-Type: application/json

Name	Description
transaction	The encoded message that is returned from the SDK

e. Response

Status	Description
200	**OK**
400	**Bad Request** The request must be corrected or added to the

	required parameters.
404	**Not Found** The requested resource was not found.
500	**Internal Server Error** An unexpected error occurred on the server.